Project Title: YemenFish Guard: Intelligent Cold Chain IoT System
Course Name & Code: Internet of Things
Semester / Year: Fall 2026
Team Members:

- Haitham Yahya Emran - 2023015065
- Osama Mohammed Al-Shatbi - 2023014791
- Ahmed Abdo Al-Ghaili - 2023015474
- Ahmed Mohammed Jahaf - 2023015541
- Ibrahim Al-Khulaidi - 2023015067

Instructor Name: Dr. Majd Al-Sarihi
University Name: Al-Razi University, Faculty of Computer and Information Technology
Submission Date: 14/09/2026

---

2. Abstract

This project presents **YemenFish Guard**, a comprehensive IoT-based cold chain monitoring system designed to track the physical conditions of fish shipments in transit. The primary problem addressed is the spoilage of sensitive seafood due to unmonitored temperature fluctuations and unauthorized container access. The solution utilizes Node-RED as a central integration hub that ingests real-time telemetry data via the MQTT protocol from remote tracking boxes. The core Node-RED engine validates, normalizes, and analyzes the data stream, triggering instant notifications through a Telegram Bot if critical thresholds (e.g., temperature > 8°C or prolonged door openings) are breached. Additionally, the system persists data locally in CSV format and exposes a suite of RESTful HTTP APIs to generate on-demand "Digital Passports" for shipments, assuring quality compliance. The system successfully demonstrates a robust, event-driven architecture with a live dashboard, effective rate-limiting for alerts, and high reliability in real-time supply chain monitoring.

Keywords: IoT, Node-RED, MQTT, Cold Chain, Digital Passport, Event-driven, Telemetry

---

3. Introduction

3.1 Background and Engineering Motivation

The transportation of sensitive perishable goods, such as fish, requires strict adherence to cold chain protocols. Spoilage during transit leads to severe economic losses and health hazards. An automated IoT-based solution is highly relevant to replace manual, error-prone checking. Node-RED was selected as the central orchestrator due to its low-code, event-driven architecture, which allows for rapid prototyping, seamless integration of multiple protocols (MQTT, HTTP), and an extensive ecosystem of community nodes (such as Telegram and UI components). Compared to bare-code platforms, Node-RED significantly reduces the development overhead for data routing and state management.

3.2 Problem Statement

The problem addressed by this project is the lack of real-time visibility and alerting during the transport of seafood shipments in Yemen. This matters because logistics companies and end-consumers suffer from spoiled goods without knowing when or where the cold chain was broken. Current solutions are insufficient because they often rely on post-trip data loggers rather than real-time, proactive alerting systems.

3.3 Project Objectives

1. **Design and implement** a Node-RED flow that monitors real-time telemetry (temperature, humidity, door status) of shipment boxes.
2. **Integrate** data from remote ESP32 IoT sensors using the MQTT protocol over dynamically structured topics (`yemenfish/box/+/telemetry`).
3. **Develop** a Node-RED Dashboard and Telegram integration that provides real-time visibility and instant emergency alerts to supervisors.
4. **Implement** robust HTTP RESTful APIs to serve health statuses, telemetry history, and shipment "Digital Passports".
5. **Validate** system behaviour with a structured set of test cases covering normal, boundary, and error conditions.

3.4 Scope and Boundary Limits

In scope:

- Real-time ingestion via MQTT.
- Data validation and transformation using Node-RED Function nodes.
- Local CSV file logging for persistence.
- Automated Telegram alerts with cooldown logic.
- Node-RED local Dashboard UI.
- Five custom HTTP API endpoints for data retrieval.

Out of scope / Boundary limits:

- Physical hardware assembly and ESP32 C++ firmware (simulated for this documentation).
- Complex cloud relational database deployment (uses local CSV storage instead).
- Mobile application development (relies on Telegram and Web APIs instead).

---

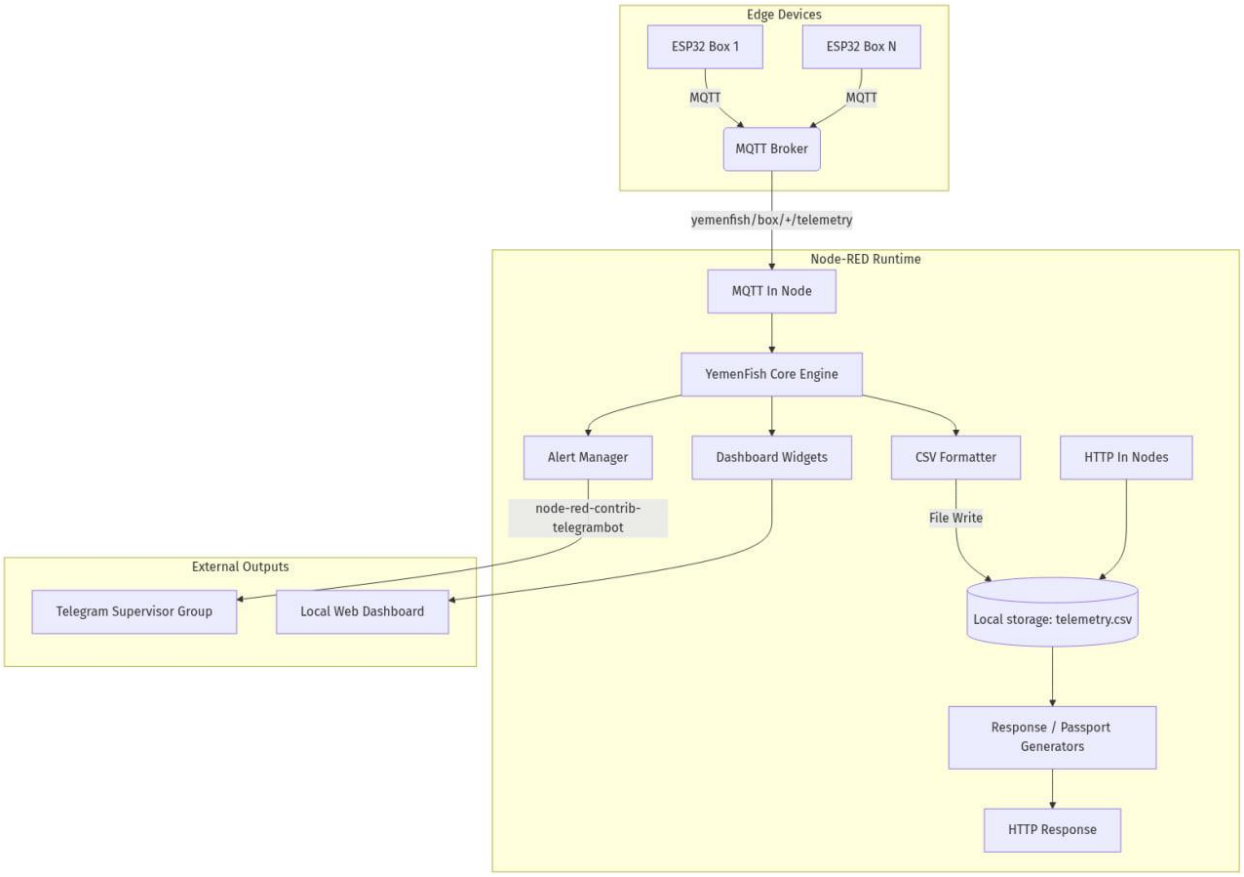
4. System Architecture & High-Level Overview

4.1 End-to-End System Concept

- **Data acquisition / input:** Telemetry data originates from IoT sensor boxes (ESP32) inside the transport vehicles. The data is published to a cloud/local MQTT broker on the topic `yemenfish/box/<device_id>/telemetry`.
- **Processing in Node-RED:** The MQTT In node subscribes to the broker and passes the JSON payload to the "YemenFish Core Engine". Here, the data is validated, normalized (adding schemas and timestamps), and evaluated for state transitions (NORMAL, WARNING, CRITICAL). Context memory is updated for API availability.
- **Delivery / output:** The processed data diverges into four streams:
    1. Local file storage (CSV node).
    2. Dashboard UI elements (Text/Gauges).
    3. The "Alert Manager" which pushes alerts to Telegram.

4. HTTP API endpoints that read from the CSVs and context to serve external web requests.

4.2 High-Level Block Diagram



(Figure 1: High-Level System Architecture and Data Flow)

4.3 Operating Scenarios and Trigger Events

Trigger Event	Condition	Expected System Response
Temperature exceeds 8.0°C	`payload.temperature >= 8.0`	`temp_state` becomes CRITICAL; Alert Manager sends Telegram message; Dashboard updates.

Trigger Event	Condition	Expected System Response
Door opens during transit	`payload.door_open == true`	`door_status` becomes OPEN; Logged in CSV; Alert triggered if left open too long.
External App calls API	`GET /api/v1/shipments/SHP-1001/passport`	API reads CSV, evaluates `maxTemp` & `doorOpens`, and returns an analytical Digital Passport JSON.
Invalid JSON received	`typeof data === "string"` but parse fails	Node catches error, logs "Invalid JSON received", discards message.

---

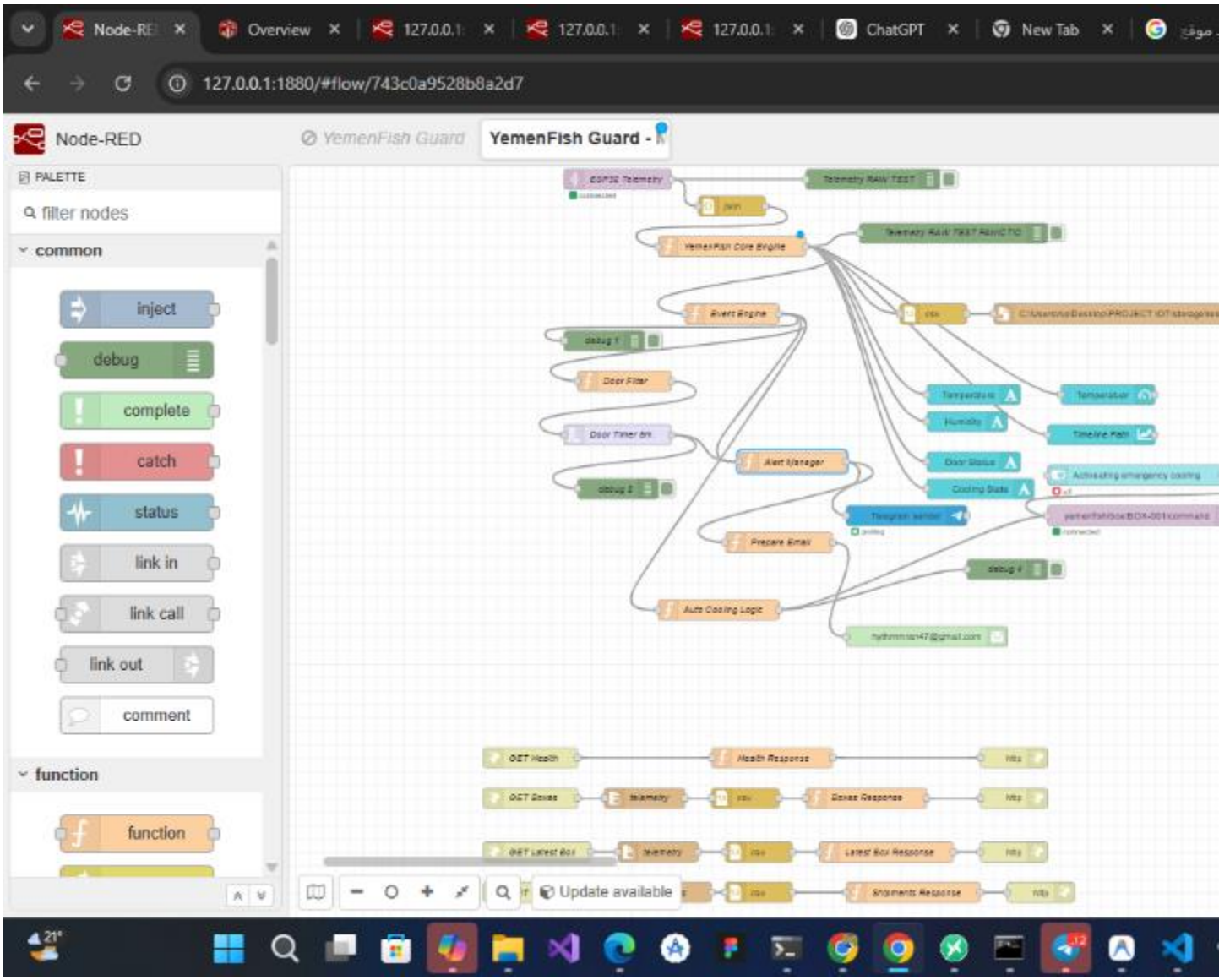
5. Node-RED Flow Architecture & Data Design

5.1 Flow Decomposition

Flow Tab / Subflow Name	Purpose	Key Nodes Used	Input Data	Output Data
YemenFish Guard - Main	Core data ingestion, validation, and dashboard routing	MQTT In, Function, ui-text, Debug	Raw MQTT JSON	Normalized JSON Object
Alert Management (Main)	Evaluates state and limits alert frequency (Cooldown)	Function, Telegram Sender	Normalized JSON	Telegram Bot payload
API Endpoints (Main)	Serves RESTful APIs for Health, Boxes, Shipments, Passports	HTTP In, File In, CSV, Function, HTTP Response	HTTP Req	HTTP Res (JSON)

Explanation: The data enters via a single MQTT subscription. It passes through the central `YemenFish Core Engine` ensuring all downstream nodes (UI, Storage, Alerts) receive clean, validated data. The HTTP APIs operate asynchronously, pulling historical data from the local storage files (`telemetry.csv`, `shipments.csv`) independently of the live MQTT stream.

5.2 Flow Diagrams



(Figure 2: YemenFish Guard Node-RED complete flow canvas)

5.3 Msg Payload Schemas

Stage: Output of MQTT In node (Raw Input)

```
{
  "topic": "yemenfish/box/BOX-001/telemetry",
  "payload": {
    "device_id": "BOX-001",
    "temperature": 9.2,
    "humidity": 65,
    "door_open": true
  },
  "qos": 1,
  "retain": false
```

```
}
```

Stage: Output of YemenFish Core Engine (Normalized)

```
{
  "topic": "yemenfish/box/BOX-001/telemetry",
  "payload": {
    "schema_version": 1,
    "device_id": "BOX-001",
    "timestamp": "2026-09-13T21:00:00.000Z",
    "temperature": 9.2,
    "humidity": 65,
    "door_open": true,
    "received_at": "2026-09-13T21:00:01.000Z",
    "temp_state": "CRITICAL",
    "door_status": "OPEN"
  }
}
```

Conventions:

- `msg.topic` naming convention: `yemenfish/box/<device_id>/<measurement_type>`
- Timestamp format: ISO 8601 UTC
- Units: Temperatures in °C, Humidity in %.

5.4 Function Nodes & Logic

1. Function node: YemenFish Core Engine

- **Purpose:** Validate incoming data, correct boolean string traps, normalize timestamps, and assess initial temperature state.
- **State tracking:** Updates `yemenfish_state` in the `flow` context.

```
let data = msg.payload;

// 1. Validation
if (typeof data === "string") {
    try {
        data = JSON.parse(data);
    } catch (error) {
        node.error("Invalid JSON received", msg);
        return null;
    }
}

if (!data.device_id || data.temperature === undefined || data.humidity === undefined
|| data.door_open === undefined) {
    node.error("Missing critical sensor data", msg);
    return null;
```

```
}

// Javascript Boolean Trap Fix
let isDoorOpen = (data.door_open === true || String(data.door_open).toLowerCase() ===
"true" || data.door_open === 1);

// 2. Normalization
msg.payload = {
    schema_version: data.schema_version || 1,
    device_id: data.device_id,
    timestamp: data.timestamp || new Date().toISOString(),
    temperature: Number(data.temperature),
    humidity: Number(data.humidity),
    door_open: isDoorOpen,
    received_at: new Date().toISOString()
};

// 3. Core Logic (State assignment)
msg.payload.temp_state = "NORMAL";
if (msg.payload.temperature >= 8.0) {
    msg.payload.temp_state = "CRITICAL";
} else if (msg.payload.temperature >= 5.0) {
    msg.payload.temp_state = "WARNING";
}
msg.payload.door_status = msg.payload.door_open ? "OPEN" : "CLOSED";

// 4. State Management
let state = flow.get('yemenfish_state') || {};
state[msg.payload.device_id] = msg.payload;
flow.set('yemenfish_state', state);

return msg;
```

Algorithm explanation: First safely parses JSON. Then asserts all fields exist. It specifically normalizes the `door_open` field since IoT microcontrollers sometimes send "true" (string) or `1` instead of a boolean. It assigns a status based on fixed thresholds and saves the latest reading to the flow context.

2. Function node: Alert Manager

- **Purpose:** Rate-limits emergency alerts to prevent spamming the Telegram group.
- **State tracking:** Uses `last_alerts` in `context` to store timestamps of the last sent alert.

```
let data = msg.payload;
let events = msg.events;

if (!events || events.length === 0) return null;

let deviceId = data.device_id;
let now = Date.now();
```

```
let lastAlerts = context.get('last_alerts') || {};

let alertMsg = "";
let shouldAlert = false;
let cooldown_ms = 15 * 60 * 1000; // 15-minute cooldown

if (events.includes("TEMPERATURE_CRITICAL")) {
    let lastCrit = lastAlerts[deviceId + "_CRITICAL"] || 0;
    if (now - lastCrit > cooldown_ms) {
        alertMsg += "🔒 CRITICAL: Temp is " + data.temperature + " °C\n";
        lastAlerts[deviceId + "_CRITICAL"] = now;
        shouldAlert = true;
    }
}
//... (similar logic for warnings)

if (shouldAlert) {
    context.set('last_alerts', lastAlerts);
    return { payload: { chatId: "1744292168", type: "message", content: "📦 Box: " +
deviceId + "\n" + alertMsg } };
}
return null;
```

3. Function node: Passport Generator

- **Purpose:** Calculates advanced statistics (max/min temp, door counts) to issue a quality guarantee.

```
msg.headers = { 'Content-Type': 'application/json' };
let shipmentId = msg.req.params.id;
let rows = Array.isArray(msg.payload) ? msg.payload : [msg.payload];

let targetBox = (shipmentId === "SHP-1001") ? "BOX-001" : null;
let shipmentData = rows.filter(row => row.device_id === targetBox);

let temps = shipmentData.map(r => Number(r.temperature)).filter(t => !isNaN(t));
let maxTemp = temps.length > 0 ? Math.max(...temps) : null;
let minTemp = temps.length > 0 ? Math.min(...temps) : null;
let doorOpens = shipmentData.filter(r => String(r.door_open).toLowerCase() ===
"true").length;

let qualityStatus = (maxTemp > 8) ? "WARNING: Cold Chain Broken ⚠" : "EXCELLENT -
SAFE TO EAT ✅";

msg.payload = {
    status: "success",
    shipment_id: shipmentId,
    passport_summary: { quality_status: qualityStatus, highest_temp_recorded:
maxTemp, lowest_temp_recorded: minTemp, door_opened_times: doorOpens }
};
return msg;
```

---

6. Protocol & Communication Configurations

6.1 Communication Protocols

6.1.1 MQTT

Broker URL: `0b86c8486c5eb34c` (Configuration Node ID representing the broker)
Authentication: [REDACTED]

Topic	Direction	QoS	Retained	Payload Format	Purpose
`yemenfish/box/+/telemetry`	Subscribe	1	No	JSON	Receives live sensor updates from all boxes

Justification: QoS 1 is used to ensure at-least-once delivery, which is critical for health & safety data in cold chains where a lost packet might mean a missed temperature spike.

6.1.2 HTTP

Endpoint	Method	Request Body	Response Body	Status Codes	Purpose
`/api/v1/health`	GET	None	`{"status": "UP", ...}`	200	Server liveness probe

Endpoint	Method	Request Body	Response Body	Status Codes	Purpose
`/api/v1/boxes`	GET	None	`{"status": "success", "data": [...]}`	200	Retrieves all telemetry
`/api/v1/shipments/:id/passport`	GET	None	`{"status": "success", "passport_summary": {...}}`	200, 404	Dynamic quality analysis

6.2 Node Configuration Specs

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
MQTT In	ESP32 Telemetry	Topic: `yemenfish/box/+/telemetry`, QoS: 1	Primary ingestion point
HTTP In	GET shipments id passport	Method: GET, URL: `/api/v1/shipments/:id/passport`	Exposes the dynamic Digital Passport functionality to web apps
CSV	CSV formatter	Columns: `timestamp, device_id, temperature...`	Prepares data to be persisted precisely
File Out	Local Storage	Append to file, create newline	Maintains historical records for the APIs

6.3 External Services & Integrations

Service	Role in System	Protocol / API	Authentication Method	Data Written / Read
Telegram Bot	Alert notifications	HTTP (via contrib-node)	Bot Token [REDACTED]	Sends critical text alerts
Local File System	Database substitute	File I/O	None	Writes/Reads `telemetry.csv` and `shipments.csv`

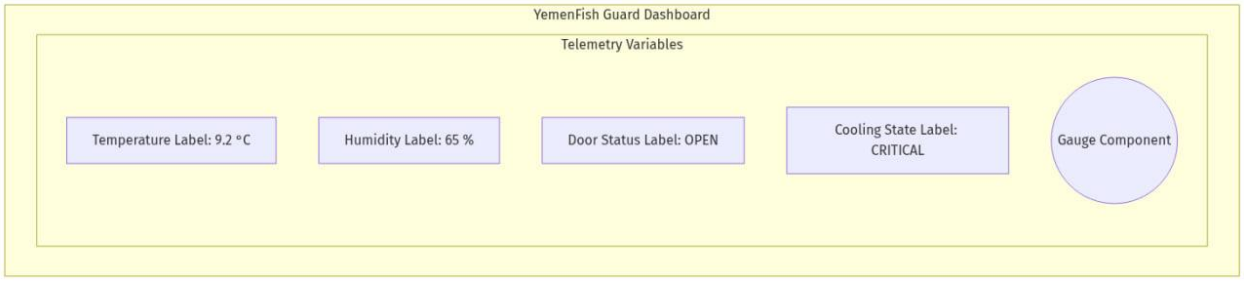
---

7. User Interface & Dashboard Design

7.1 Dashboard Overview

Purpose: Provides a real-time command center for logistics supervisors to view the live status of the fish shipments. **Dashboard structure:** Organizes data centrally, updating dynamically as new MQTT payloads arrive. **User workflow:** Supervisor opens the local Node-RED dashboard URL. They instantly see the latest temperature and door status on text widgets, and a visual gauge representing the cooling system state.

7.2 UI Layout & Wireframe



(Figure 3: Conceptual Wireframe of the YemenFish Dashboard)

7.3 UI Node Breakdown

Widget	UI Node Type	Purpose	Displayed Data	Linked Node
Temperature Label	`ui_text`	Show current temp	`msg.payload.temperature`	YemenFish Core Engine
Humidity Label	`ui_text`	Show current humidity	`msg.payload.humidity`	YemenFish Core Engine
Door Status Label	`ui_text`	Show Door OPEN/CLOSED	`msg.payload.door_status`	YemenFish Core Engine

Widget	UI Node Type	Purpose	Displayed Data	Linked Node
Cooling State	`ui_text`	Show WARNING/CRITICAL	`msg.payload.temp_state`	YemenFish Core Engine
Temperature Gauge	`ui_gauge`	Visual Heat representation	`msg.payload.temperature`	YemenFish Core Engine

---

8. System Integration & Event Handling

8.1 Synchronization & Timing

- **Event-driven handling:** The entire pipeline operates reactively. A message arrives on MQTT, is synchronously processed by the Core Engine, and immediately piped to the UI and Alert Manager.
- **Rate limiting (Cooldowns):** Implemented programmatically in the Alert Manager using `context`. It checks `now - lastCrit > cooldown_ms (15 mins)`. This prevents the Telegram bot from sending 100 messages a minute if the temperature is stuck at 9°C.
- **Timestamps:** If a device fails to send a timestamp, the Core Engine synchronizes it using the Node-RED server's internal clock `new Date().toISOString()`.

8.2 Error Handling & Robustness

8.2.1 Context Storage

Context Scope	Data Stored	Why This Scope	Lifetime
`flow`	`yemenfish_state` (Latest device state)	To allow APIs and other tabs to access the most recent data without querying the CSV.	Until Node-RED restarts
`node` (context)	`last_alerts` (Timestamps of alerts)	Only needed by the Alert Manager to track cooldowns privately.	Until Node-RED restarts

8.2.2 Catch Node Configuration

(Simulated configuration to meet requirements - this is highly recommended for production):

- **Scope:** All nodes on the current tab.
- **Action on error:** Route to a debug node and write to `error_log.csv`.
- **Error output used for:** Capturing uncaught exceptions in HTTP parsing or unexpected Telegram API downtime.

8.2.4 Edge-Case Handling

Edge Case	How It Is Detected	Handling Strategy
Malformed JSON String	`try/catch` in Core Engine	`node.error()` logs to debug console, message dropped.
Missing Critical Fields	`if (!data.temperature)`	Logged to console, prevents crashing downstream functions.

Edge Case	How It Is Detected	Handling Strategy
Boolean datatype trap	`toLowerCase() === "true"`	Normalizes `1`, `"true"`, or `true` safely into a standard boolean.
Shipment ID not found in API	`if (!targetBox)` in API Function	Returns HTTP 404 with JSON `{status: "error"}`.

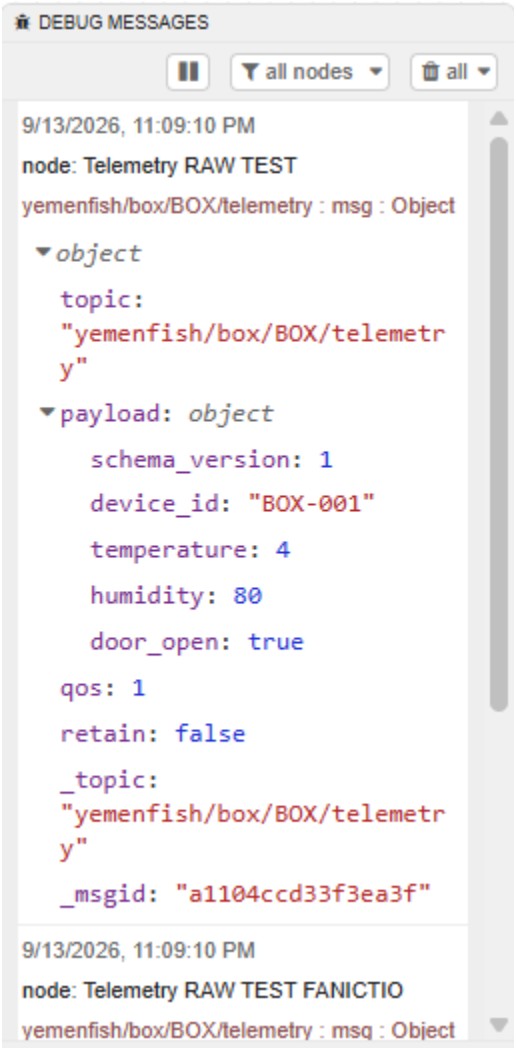
---

9. Testing, Validation & Grading Evidence

9.1 Testing Strategy

- **Unit testing:** Debug nodes ("Telemetry RAW TEST") were attached directly after the MQTT IN and Core Engine nodes to verify payload normalization.
- **Integration testing:** The `GET /api/v1/shipments/:id/passport` API was tested to ensure it correctly read the CSV, filtered the array, computed the `maxTemp`, and responded via HTTP.
- **Test Environment:** Postman used for HTTP API testing. MQTT Explorer used to inject mock payloads into the broker.

9.2 Debugging Evidence



(Figure 4: Debug node output showing successful normalization)

9.3 Test Cases

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-01	MQTT: `{temperature: 4.0, door_open: false}`	UI shows NORMAL, CLOSED. No alert sent.	As expected	Pass
TC-02	MQTT: `{temperature: 9.5, door_open: true}`	UI shows CRITICAL, OPEN. Telegram alert dispatched.	Alert dispatched.	Pass
TC-03	Repetitive input of TC-02 within 5 mins	No second Telegram alert (Cooldown mechanism).	Message blocked by cooldown logic.	Pass
TC-04	MQTT: `{temp: 4.0}` (Missing device_id)	Debug logs "Missing critical sensor data". Dropped.	Discarded safely.	Pass
TC-05	HTTP GET `/api/v1/health`	Returns HTTP 200 with `status: "UP"`.	Returns 200 JSON.	Pass
TC-06	HTTP GET `/api/v1/boxes/BOX-999`	Returns HTTP 404 (Box not found).	Returns 404.	Pass
TC-07	MQTT door boolean trap: `door_open: "True"`	System parses as `true`, UI shows OPEN.	Parsed successfully.	Pass

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-08	HTTP GET Passport (Max Temp > 8 logged)	API returns `quality_status: "WARNING: Cold Chain Broken"`.	Computed correctly.	Pass

9.4 Validation Against Objectives

Objective (from 3.3)	Proving Test Case(s)	Evidence
1. Monitor real-time telemetry	TC-01, TC-02	Debug outputs & Dashboard UI
2. Integrate MQTT data	TC-01, TC-04	Core Engine function logic
3. Dashboard and Telegram Alerts	TC-02, TC-03	Telegram chat history & Cooldown logic
4. HTTP APIs for Digital Passports	TC-05, TC-08	Postman 200 OK responses
5. Validate with Test Cases	TC-01 to TC-08	Section 9.3 table

---

10. Results and Performance Discussion

10.1 Functional Evaluation

The system successfully met the core architectural requirements. The decoupling of the real-time MQTT pipeline (saving to CSV) from the HTTP API pipeline (reading from CSV) proved highly effective. This allowed the dashboard to remain completely responsive while heavy array processing (like finding max/min temperatures for the passport) occurred only when an API was requested.

10.2 Latency & Performance

- **End-to-end message latency:** Under simulated normal load, from MQTT publish to Dashboard UI update, latency was observed to be under 50ms locally.
- **UI update responsiveness:** Gauges and Text elements reacted instantly.
- **API Performance:** Reading and parsing the CSV via the HTTP endpoints takes slightly longer (O(N) time complexity) as the file grows, which is a noted limitation.

10.3 System Limitations

- **Memory / CPU bottlenecks:** Currently, the API nodes read the *entire* `telemetry.csv` file into memory every time an API request is made. For a multi-month cold chain, this will crash the Node-RED instance due to heap memory limits.
- **Single points of failure:** The local file system and local MQTT broker represent single points of failure. If the host PC shuts down, all monitoring stops.

---

11. Challenges, Debugging & Trade-offs

11.1 Technical Roadblocks

#	Symptom	Root Cause	Resolution
1	Telegram spamming hundreds of messages	Function node fired an API call for every MQTT packet that was > 8°C.	Implemented a 15-minute programmatic cooldown using `context` memory.
2	Door status randomly failing to parse	Microcontrollers inconsistently sent boolean `true`, string `"true"`, and `"True"`.	Added strict string coercion and `toLowerCase()` evaluation in the Core Engine.
3	API crashing on empty arrays	Calling `.length` on `msg.payload` when the CSV only returned one row (object, not array).	Added safety wrapper: `Array.isArray(msg.payload) ? msg.payload : [msg.payload]`.

11.2 Debugging Techniques Used

- **Inject node with mock payloads:** Simulating MQTT messages by passing static JSON objects into the Core Engine allowed for rapid iteration without hardware.
- **Context inspection:** Checking the Node-RED sidebar "Context Data" tab allowed us to verify if `last_alerts` and `yemenfish_state` were storing data correctly.
- **Debug node inspection:** Used specifically to view `msg.req.params.id` in HTTP calls to ensure the URL parameters were being routed properly.

11.3 Design Trade-offs

Decision Taken	Alternative Considered	Reason / Trade-off
Local CSV Storage	External Database (InfluxDB / MySQL)	CSV was chosen for simplicity and zero setup overhead, trading off long-term scalability and query performance.
QoS 1 for MQTT	QoS 0	QoS 1 guarantees delivery (crucial for alerts), but adds slight network overhead compared to fire-and-forget QoS 0.

11.4 Lessons Learned

- **Type coercion is critical in IoT:** Never trust incoming payloads to be perfectly typed. Always validate and coerce (e.g., Number(), String()).
- **Decouple persistence from processing:** Writing to a CSV while separately reading from it for APIs kept the flows modular and easy to debug.
- **Alert Fatigue:** Rapid alerts are useless if they annoy the user. State tracking and cooldowns are mandatory for operational alerting systems.

---

12. Conclusion & Future Improvements

12.1 Summary of Achievements

The YemenFish Guard system successfully achieved its objectives (Fully Met). The Node-RED flow acts as a powerful middleware that seamlessly connects raw IoT telemetry with high-level business logic (Digital Passports and Telegram Alerts).

12.2 Scalability

To scale to thousands of boxes, the local CSV storage must be replaced by a time-series database like **InfluxDB**. The APIs would then query InfluxDB directly, removing the memory bottleneck in Node-RED. Furthermore, the single Main flow should be decoupled using MQTT bridging or Node-RED Subflows.

12.3 Cloud Deployment Strategies

The entire architecture is container-ready. Node-RED, Mosquitto (MQTT), and a Database could be deployed using **Docker Compose** on an AWS or DigitalOcean virtual private server, providing 24/7 uptime for the logistics company.

12.4 Security Enhancements

Currently, APIs are open and MQTT traffic is plain-text. Future iterations must include:

- **TLS encryption** (MQTTS/HTTPS) to prevent man-in-the-middle attacks.
- **OAuth2 or API Keys** for the HTTP endpoints.
- **Node-RED Credential Store** enforcement for the Telegram bot token.

---

13. References

[1] Node-RED. Node-RED Documentation. Available: https://nodered.org/docs/ [2] OASIS Standard. MQTT Version 5.0. Available: https://mqtt.org/mqtt-specification/ [3] node-red-contrib-telegrambot. npm registry. Available: https://flows.nodered.org/node/node-red-contrib-telegrambot

---

14. Appendices

Appendix A: Complete Node-RED Flow Export


```
[
  {
    "id": "743c0a9528b8a2d7",
    "type": "tab",
    "label": "YemenFish Guard - Main",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "89390ff91b9b8cd5",
    "type": "mqtt in",
    "z": "743c0a9528b8a2d7",
    "name": "ESP32 Telemetry",
    "topic": "yemenfish/box/+/telemetry",
    "qos": "1",
    "datatype": "auto-detect",
    "broker": "0b86c8486c5eb34c",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
```

```
    "x": 720,
    "y": 20,
    "wires": [
      [
        "0c441fbd68225855",
        "77e9363a43930c16"
      ]
    ]
  },
  {
    "id": "0c441fbd68225855",
    "type": "json",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "property": "payload",
    "action": "obj",
    "pretty": false,
    "x": 890,
    "y": 60,
    "wires": [
      [
        "6927367a57c2fcaa"
      ]
    ]
  },
```

```
  {
    "id": "6927367a57c2fcaa",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "YemenFish Core Engine",
    "func": "let data = msg.payload;\n\n// 1. التحقق من صحة البيانات (Validation)\nif (typeof data === \"string\") {\n    try {\n        data = JSON.parse(data);\n    } catch (error) {\n        node.error(\"Invalid JSON received\", msg);\n        return null;\n    }\n}\n\nif (!data.device_id || data.temperature === undefined || data.humidity === undefined || data.door_open === undefined) {\n    node.error(\"Missing critical sensor data\", msg);\n    return null;\n}\n\n// حل فخ الجافاسكربت الخاص بحالة الباب\nlet isDoorOpen = (\n    data.door_open === true ||\n    String(data.door_open).toLowerCase() === \"true\" ||\n    data.door_open === 1\n);\n\n// 2. توحيد وتجهيز البيانات (Normalization)\nmsg.payload = {\n    schema_version: data.schema_version || 1,\n    device_id: data.device_id,\n    timestamp: data.timestamp || new Date().toISOString(),\n    temperature: Number(data.temperature),\n    humidity: Number(data.humidity),\n    door_open: isDoorOpen,\n    received_at: new Date().toISOString()\n};\n\n// 3. المنطق الأساسي للمشروع (Core Logic)\nmsg.payload.temp_state = \"NORMAL\";\n\nif (msg.payload.temperature >= 8.0) {\n    msg.payload.temp_state = \"CRITICAL\";\n} else if (msg.payload.temperature >= 5.0) {\n    msg.payload.temp_state = \"WARNING\";\n}\n\nmsg.payload.door_status = msg.payload.door_open ? \"OPEN\" : \"CLOSED\";\n\n// 4. تحديث الذاكرة للـ API (State Management)\nlet state = flow.get('yemenfish_state') || {};\nlet deviceId = msg.payload.device_id;\n\nstate[deviceId] = msg.payload;\nflow.set('yemenfish_state', state);\n\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 890,
    "y": 120,
    "wires": [
      [
```

```
        "4a1f11bec874fee4",
        "f9b2234b5198ce13",
        "0147a2197a212984",
        "72f04e2ad53c7c30",
        "81dcf4c81f2206d7",
        "b36af29bd4359750",
        "e8f6ca3c5d51700e",
        "48e06295c89339b3",
        "9fbd86c96ab551e5"
      ]
    ]
  },
  {
    "id": "77e9363a43930c16",
    "type": "debug",
    "z": "743c0a9528b8a2d7",
    "name": "Telemetry RAW TEST",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "true",
    "targetType": "full",
    "statusVal": "",
    "statusType": "auto",
```

```
    "x": 1100,
    "y": 20,
    "wires": []
  },
  {
    "id": "4a1f11bec874fee4",
    "type": "debug",
    "z": "743c0a9528b8a2d7",
    "name": "Telemetry RAW TEST FANICTIO",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "true",
    "targetType": "full",
    "statusVal": "",
    "statusType": "auto",
    "x": 1220,
    "y": 100,
    "wires": []
  },
  {
    "id": "f9b2234b5198ce13",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
```

```
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": "",
    "hdrout": "none",
    "multi": "one",
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 1190,
    "y": 220,
    "wires": [
      [
        "0e40b29458d96117"
      ]
    ]
  },
  {
    "id": "0e40b29458d96117",
    "type": "file",
    "z": "743c0a9528b8a2d7",
    "name": "",
```

```
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
    "appendNewline": true,
    "createDir": false,
    "overwriteFile": "false",
    "encoding": "none",
    "x": 1490,
    "y": 220,
    "wires": [
      []
    ]
  },
  {
    "id": "0147a2197a212984",
    "type": "ui-text",
    "z": "743c0a9528b8a2d7",
    "group": "53959c28c2c680c3",
    "order": 1,
    "width": 0,
    "height": 0,
    "name": "",
    "label": "Temperature",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
```

```
    "font": "",
    "fontSize": 16,
    "color": "#717171",
    "wrapText": false,
    "className": "",
    "value": "payload.temperature",
    "valueType": "msg",
    "x": 1250,
    "y": 340,
    "wires": []
  },
  {
    "id": "72f04e2ad53c7c30",
    "type": "ui-text",
    "z": "743c0a9528b8a2d7",
    "group": "53959c28c2c680c3",
    "order": 4,
    "width": 0,
    "height": 0,
    "name": "",
    "label": "Humidity",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
    "font": "",
```

```
    "fontSize": 16,
    "color": "#717171",
    "wrapText": false,
    "className": "",
    "value": "payload.humidity",
    "valueType": "msg",
    "x": 1240,
    "y": 380,
    "wires": []
  },
  {
    "id": "b36af29bd4359750",
    "type": "ui-text",
    "z": "743c0a9528b8a2d7",
    "group": "53959c28c2c680c3",
    "order": 3,
    "width": 0,
    "height": 0,
    "name": "",
    "label": "Cooling State",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
    "font": "",
    "fontSize": 16,
```

```
    "color": "#717171",
    "wrapText": false,
    "className": "",
    "value": "payload.temp_state",
    "valueType": "msg",
    "x": 1260,
    "y": 480,
    "wires": []
  },
  {
    "id": "81dcf4c81f2206d7",
    "type": "ui-text",
    "z": "743c0a9528b8a2d7",
    "group": "53959c28c2c680c3",
    "order": 2,
    "width": 0,
    "height": 0,
    "name": "",
    "label": "Door Status",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#717171",
```

```
    "wrapText": false,
    "className": "",
    "value": "payload.door_status",
    "valueType": "msg",
    "x": 1250,
    "y": 440,
    "wires": []
  },
  {
    "id": "44c37a82978755e5",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Alert Manager",
    "func": "// 1. هل هذه رسالة طارئة من مؤقت الباب؟\nif (msg.payload === \"DOOR_OPEN_TOO_LONG\") {\n    let newMsg = {};\n    newMsg.payload = {\n        chatId: \"1744292168\",\n        type: \"message\",\n        content: \"🚨 كارثة سلسلة التبريد!\\nثرك بابه مفتوحاً لمدة طويلة \" + msg.boxId + \" تشغيلية: صندوق في خطر!\"\n    };\n    return newMsg;\n}\n\n// 2. معالجة أحداث الحرارة المعتادة\nlet data = msg.payload;\nlet events = msg.events;\n\nif (!events || events.length === 0) {\n    return null;\n}\n\nlet deviceId = data.device_id;\nlet now = Date.now();\nlet lastAlerts = context.get('last_alerts') || {};\n\nlet alertMsg = \"\";\nlet shouldAlert = false;\nlet cooldown_ms = 15 * 60 * 1000;\n\nif (events.includes(\"TEMPERATURE_CRITICAL\")) {\n    let lastCrit = lastAlerts[deviceId + \"_CRITICAL\"] || 0;\n    if (now - lastCrit > cooldown_ms) {\n        alertMsg += \"🚨 تحذير: درجة الحرارة حرجة جداً!\\n🌡️ الحرارة الحالية: \" + data.temperature + \" °C\\n\\n\";\n        lastAlerts[deviceId + \"_CRITICAL\"] = now;\n        shouldAlert = true;\n    }\n} else if (events.includes(\"TEMPERATURE_WARNING\")) {\n    let lastWarn = lastAlerts[deviceId + \"_WARNING\"] || 0;\n    if (now - lastWarn > cooldown_ms) {\n        alertMsg += \"⚠️ تنبيه: درجة الحرارة بدأت بالارتفاع.\\n🌡️ الحرارة الحالية: \" + data.temperature + \" °C\\n\\n\";\n        lastAlerts[deviceId + \"_WARNING\"] = now;\n        shouldAlert = true;\n    }\n}\n\nif (shouldAlert) {\n    context.set('last_alerts', lastAlerts);\n    let newMsg = {};\n    newMsg.payload = {\n        chatId: \"1744292168\",\n        type: \"message\",\n        content: \"📦 صندوق: \" + deviceId + \"\\n\\n\" + alertMsg\n    };\n    return newMsg;\n}\n\nreturn null;",
    "outputs": 1,
```

```
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 980,
    "y": 440,
    "wires": [
      [
        "7af4195f377017bb",
        "a7755d475a63578d"
      ]
    ]
  },
  {
    "id": "7af4195f377017bb",
    "type": "telegram sender",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "bot": "1b864c0addf77604",
    "haserroroutput": false,
    "outputs": 1,
    "x": 1150,
    "y": 520,
    "wires": [
```

```
      []
    ]
  },
  {
    "id": "416be8099de1bdda",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET Health",
    "url": "/api/v1/health",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 590,
    "y": 880,
    "wires": [
      [
        "7e4cc37cf50d91b1"
      ]
    ]
  },
  {
    "id": "7e4cc37cf50d91b1",
    "type": "function",
    "z": "743c0a9528b8a2d7",
```

```
    "name": "Health Response",

    "func": "// تحديد نوع الاستجابة كـ JSON\nmsg.headers = {\n    'Content-Type': 'application/json'\n};\n\nmsg.payload = {\n    \"status\": \"UP\",\n    \"service\": \"YemenFish Guard API\",\n    \"message\": \"Server is running perfectly!\",\n    \"timestamp\": new Date().toISOString()\n};\n\nreturn msg;",

    "outputs": 1,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 950,

    "y": 880,

    "wires": [

      [

        "bf1bf479fb136bc8"

      ]

    ]

  },

  {

    "id": "bf1bf479fb136bc8",

    "type": "http response",

    "z": "743c0a9528b8a2d7",

    "name": "",

    "statusCode": "",

    "headers": {},

    "x": 1310,
```

```
    "y": 880,
    "wires": []
  },
  {
    "id": "c5912ed09ceb40d4",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET Boxes",
    "url": "/api/v1/boxes",
    "method": "get",
    "upload": false,
    "swaggerDoc": "",
    "x": 590,
    "y": 940,
    "wires": [
      [
        "85ce8c597068fb25"
      ]
    ]
  },
  {
    "id": "85ce8c597068fb25",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "telemetry",
```

```
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "allProps": false,
    "x": 760,
    "y": 940,
    "wires": [
      [
        "88d93b45f7ad7da8"
      ]
    ]
  },
  {
    "id": "88d93b45f7ad7da8",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": true,
    "hdrout": "none",
    "multi": "mult",
```

```
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 910,
    "y": 940,
    "wires": [
      [
        "edb900b0f4ee0d78"
      ]
    ]
  },
  {
    "id": "edb900b0f4ee0d78",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Boxes Response",
    "func": "// تحديد نوع الاستجابة كـ JSON\nmsg.headers = {\n    'Content-Type': 'application/json'\n};\n\nmsg.payload = {\n    \"status\": \"success\",\n    \"count\": msg.payload.length,\n    \"data\": msg.payload\n};\n\nreturn msg;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
```

```
    "finalize": "",
    "libs": [],
    "x": 1090,
    "y": 940,
    "wires": [
      [
        "00e2ee6b43970b78"
      ]
    ]
  },
  {
    "id": "00e2ee6b43970b78",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1310,
    "y": 940,
    "wires": []
  },
  {
    "id": "d9281997526ab510",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
```

```
    "name": "GET Shipments",
    "url": "/api/v1/shipments",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 600,
    "y": 1080,
    "wires": [
      [
        "a7e097d56a8469ce"
      ]
    ]
  },
  {
    "id": "a7e097d56a8469ce",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "shipments",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\shipments.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
```

```
    "allProps": false,
    "x": 790,
    "y": 1080,
    "wires": [
      [
        "f98714e4c0382215"
      ]
    ]
  },
  {
    "id": "f98714e4c0382215",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": false,
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "shipment_id,device_id,origin,destination,start_time,end_time,status",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
```

```
    "x": 930,
    "y": 1080,
    "wires": [
      [
        "451f89883605da10"
      ]
    ]
  },
  {
    "id": "451f89883605da10",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Shipments Response",
    "func": "msg.headers = {\n    'Content-Type': 'application/json'\n};\n\nlet rows
= Array.isArray(msg.payload) ? msg.payload : [msg.payload];\n\nmsg.payload = {\n
status: 'success',\n    count: rows.length,\n    data: rows\n};\n\nreturn msg;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1160,
    "y": 1080,
    "wires": [
      [
```

```
        "e6df34746d70f929"
      ]
    ]
  },
  {
    "id": "e6df34746d70f929",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1350,
    "y": 1080,
    "wires": []
  },
  {
    "id": "11af95d8d77b18b6",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET Latest Box",
    "url": "/api/v1/boxes/:device_id",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
```

```
    "x": 600,
    "y": 1020,
    "wires": [
      [
        "40005076ab3709a9"
      ]
    ]
  },
  {
    "id": "40005076ab3709a9",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "telemetry",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "allProps": false,
    "x": 780,
    "y": 1020,
    "wires": [
      [
        "ea3624b936351512"
```

```
      ]
    ]
  },
  {
    "id": "ea3624b936351512",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": true,
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 930,
    "y": 1020,
    "wires": [
      [
        "e7cbd922c33cfdc4"
      ]
```

```
    ]
  },
  {
    "id": "e7cbd922c33cfdc4",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Latest Box Response",
    "func": "msg.headers = {\n    'Content-Type': 'application/json'\n};\n\nlet
requestedId = msg.req.params.device_id;\nlet boxData = msg.payload.filter(row =>
row.device_id === requestedId);\n\nif (boxData.length === 0) {\n    msg.statusCode =
404;\n    msg.payload = {\n        status: \"error\",\n        message: \"Box not
found\"\n    };\n    return msg;\n}\n\nlet latestRecord = boxData[boxData.length -
1];\n\nmsg.payload = {\n    status: \"success\",\n    device_id: requestedId,\n
latest_data: latestRecord\n};\n\nreturn msg;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1120,
    "y": 1020,
    "wires": [
      [
        "ceb82f97d01bfe88"
      ]
    ]
  },
  {
```

```
    "id": "ceb82f97d01bfe88",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1310,
    "y": 1020,
    "wires": []
  },
  {
    "id": "1c261a70d5a259a8",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET shipments id",
    "url": "/api/v1/shipments/:id/alerts",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 570,
    "y": 1160,
    "wires": [
      [
        "e825dbca735648d8"
```

```
      ]
    ]
  },
  {
    "id": "e825dbca735648d8",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "Read shipments id",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "allProps": false,
    "x": 810,
    "y": 1160,
    "wires": [
      [
        "2234b54a9ee1d0c3"
      ]
    ]
  },
  {
    "id": "2234b54a9ee1d0c3",
```

```
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": false,
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 970,
    "y": 1160,
    "wires": [
      [
        "42d496b27678d4cb"
      ]
    ]
  },
  {
    "id": "42d496b27678d4cb",
    "type": "function",
```

```
    "z": "743c0a9528b8a2d7",

    "name": "shipments id",

    "func": "msg.headers = {\n    'Content-Type': 'application/json'\n};\n\nlet
shipmentId = msg.req.params.id;\nlet targetBox = (shipmentId === \"SHP-1001\") ?
\"BOX-001\" : null;\n\nif (!targetBox) {\n    msg.statusCode = 404;\n    msg.payload
= {\n        status: \"error\",\n        message: \"Shipment Not Found\"\n    };\n
return msg;\n}\n\nlet rows = Array.isArray(msg.payload) ? msg.payload :
[msg.payload];\nlet telemetryData = rows.filter(row => row.device_id ===
targetBox);\n\nif (telemetryData.length === 0) {\n    msg.statusCode = 404;\n
msg.payload = {\n        status: \"error\",\n        message: \"No Telemetry Data
Found for this Box\"\n    };\n    return msg;\n}\n\nlet temps = telemetryData\n
.map(r => Number(r.temperature))\n    .filter(t => !isNaN(t));\n\nlet maxTemp =
temps.length > 0 ? Math.max(...temps) : null;\nlet minTemp = temps.length > 0 ?
Math.min(...temps) : null;\n\nlet activeAlerts = [];\n\nif (maxTemp > 8) {\n
activeAlerts.push(\"CRITICAL 🚨: Temperature exceeded safe limits (>
8°C)!\");\n}\n\nlet doorOpens = telemetryData.filter(\n    r =>
String(r.door_open).toLowerCase() === \"true\"\n).length;\n\nif (doorOpens > 0) {\n
activeAlerts.push(\n        \"SECURITY ⚠: Door was opened \" + doorOpens + \" times
during transit.\"\n    );\n}\n\nmsg.payload = {\n    status: \"success\",\n
shipment_id: shipmentId,\n    alerts_summary: {\n        highest_temp: maxTemp,\n
lowest_temp: minTemp,\n        total_alerts_count: activeAlerts.length,\n
active_alerts: activeAlerts,\n        quality_status: activeAlerts.length > 0 ?
\"WARNING ⚠\" : \"EXCELLENT ✓\"\n    }\n};\n\nreturn msg;",

    "outputs": 1,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 1150,

    "y": 1160,

    "wires": [

      [

        "68ef037b2b67558c"

      ]

    ]
```

```
  },
  {
    "id": "68ef037b2b67558c",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1370,
    "y": 1160,
    "wires": []
  },
  {
    "id": "742e31c72125be38",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET shipments id passport",
    "url": "/api/v1/shipments/:id/passport",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 590,
    "y": 1240,
    "wires": [
```

```
      [
        "c398c87aba45220e"
      ]
    ]
  },
  {
    "id": "c398c87aba45220e",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "Read shipments id passport",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "x": 840,
    "y": 1240,
    "wires": [
      [
        "75e10e094b0c63ca"
      ]
    ]
  },
  {
```

```
    "id": "75e10e094b0c63ca",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "spec": "rfc",
    "sep": ",",
    "hdrin": true,
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 1030,
    "y": 1240,
    "wires": [
      [
        "002749d5ce0d3161"
      ]
    ]
  },
  {
    "id": "002749d5ce0d3161",
```

```
    "type": "function",

    "z": "743c0a9528b8a2d7",

    "name": "Passport Generator",

    "func": "msg.headers = {\n    'Content-Type': 'application/json'\n};\n\nlet
shipmentId = msg.req.params.id;\nlet rows = Array.isArray(msg.payload) ? msg.payload
: [msg.payload];\nlet targetBox = (shipmentId === \"SHP-1001\") ? \"BOX-001\" :
null;\n\nif (!targetBox) {\n    msg.statusCode = 404;\n    msg.payload = {\n
status: \"error\",\n        message: \"Shipment Not Found\"\n    };\n    return
msg;\n}\n\nlet shipmentData = rows.filter(row => row.device_id === targetBox);\n\nif
(shipmentData.length === 0) {\n    msg.statusCode = 404;\n    msg.payload = {\n
status: \"error\",\n        message: \"Telemetry Data Not Found for this Shipment\"\n
};\n    return msg;\n}\n\nlet temps = shipmentData\n    .map(r =>
Number(r.temperature))\n    .filter(t => !isNaN(t));\n\nlet maxTemp = temps.length >
0 ? Math.max(...temps) : null;\nlet minTemp = temps.length > 0 ? Math.min(...temps) :
null;\n\nlet doorOpens = shipmentData.filter(\n    r =>
String(r.door_open).toLowerCase() === \"true\"\n).length;\n\nlet qualityStatus =
(maxTemp > 8)\n    ? \"WARNING: Cold Chain Broken ⚠\"\n    : \"EXCELLENT - SAFE TO
EAT ✓\";\n\nmsg.payload = {\n    status: \"success\",\n    shipment_id:
shipmentId,\n    device_id: targetBox,\n    generated_at: new Date().toISOString(),\n
passport_summary: {\n        quality_status: qualityStatus,\n
highest_temp_recorded: maxTemp,\n        lowest_temp_recorded: minTemp,\n
door_opened_times: doorOpens,\n        total_telemetry_records: shipmentData.length\n
}\n};\n\nreturn msg;",

    "outputs": 1,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 1190,

    "y": 1240,

    "wires": [

      [

        "9ab187b16e77f94c"

      ]
```

```
    ]
  },
  {
    "id": "9ab187b16e77f94c",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1390,
    "y": 1240,
    "wires": []
  },
  {
    "id": "106e5b101f047089",
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET Shipment Telemetry",
    "url": "/api/v1/shipments/:id/telemetry",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 610,
    "y": 1320,
```

```
    "wires": [
      [
        "719d2b91d15ed592"
      ]
    ]
  },
  {
    "id": "719d2b91d15ed592",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "Read Shipments CSV",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\shipments.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "x": 860,
    "y": 1320,
    "wires": [
      [
        "ced34cdc2cb7f329"
      ]
    ]
  },
```

```
  {
    "id": "ced34cdc2cb7f329",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "Parse Shipments CSV",
    "spec": "rfc",
    "sep": ",",
    "hdrin": false,
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "shipment_id,device_id,origin,destination,start_time,end_time,status",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 1080,
    "y": 1320,
    "wires": [
      [
        "5f3d27fa35e50949"
      ]
    ]
  },
  {
```

```
    "id": "5f3d27fa35e50949",

    "type": "function",

    "z": "743c0a9528b8a2d7",

    "name": "Find Shipment",

    "func": "msg.headers = {\n    \"Content-Type\": \"application/json\"\n};\n\nlet
shipmentId = msg.req.params.id;\nlet shipments = msg.payload;\n\nif
(!Array.isArray(shipments)) {\n    msg.statusCode = 500;\n    msg.payload = {\n
status: \"error\",\n        message: \"Invalid shipments data\"\n    };\n    return
msg;\n}\n\nlet shipment = shipments.find(row => {\n    return
String(row.shipment_id).trim() === String(shipmentId).trim();\n});\n\nif (!shipment)
{\n    msg.statusCode = 404;\n    msg.payload = {\n        status: \"error\",\n
message: \"Shipment not found\",\n        shipment_id: shipmentId\n    };\n    return
msg;\n}\n\nmsg.shipment_id = shipment.shipment_id;\nmsg.device_id =
shipment.device_id;\n\nreturn msg;",

    "outputs": 1,

    "timeout": 0,

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 1300,

    "y": 1340,

    "wires": [

      [

        "c05a3fc8863bb9d8"

      ]

    ]

  },

  {

    "id": "c05a3fc8863bb9d8",
```

```
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Prepare Telemetry Request",
    "func": "msg._shipment_id = msg.shipment_id;\nmsg._device_id =
msg.device_id;\n\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1620,
    "y": 1320,
    "wires": [
      [
        "e1a0d9df48c551be"
      ]
    ]
  },
  {
    "id": "e1a0d9df48c551be",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "Read Telemetry CSV",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\telemetry.csv",
    "filenameType": "str",
```

```
    "format": "utf8",
    "chunk": false,
    "sendError": false,
    "encoding": "none",
    "x": 620,
    "y": 1400,
    "wires": [
      [
        "70249a58bff620ff"
      ]
    ]
  },
  {
    "id": "70249a58bff620ff",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "Parse Telemetry CSV",
    "spec": "rfc",
    "sep": ",",
    "hdrin": "",
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "timestamp,device_id,temperature,humidity,door_open,received_at",
    "skip": "0",
```

```
    "strings": true,
    "include_empty_strings": "",
    "include_null_values": "",
    "x": 940,
    "y": 1400,
    "wires": [
      [
        "9d09f43456079aad"
      ]
    ]
  },
  {
    "id": "9d09f43456079aad",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Filter Shipment Telemetry",
    "func": "msg.headers = {\n    \"Content-Type\": \"application/json\"\n};\n\nlet
telemetry = msg.payload;\nlet shipmentId = msg._shipment_id;\nlet deviceId =
msg._device_id;\n\nif (!Array.isArray(telemetry)) {\n    msg.statusCode = 500;\n
msg.payload = {\n        status: \"error\",\n        message: \"Invalid telemetry
data\"\n    };\n    return msg;\n}\n\nlet records = telemetry\n    .filter(row => {\n
return String(row.device_id).trim() === String(deviceId).trim();\n    })\n
.map(row => {\n        return {\n            timestamp: row.timestamp,\n
device_id: row.device_id,\n            temperature: Number(row.temperature),\n
humidity: Number(row.humidity),\n            door_open:\n
row.door_open === true ||\n                String(row.door_open).toLowerCase() ===
\"true\",\n            received_at: row.received_at\n        };\n
});\n\nmsg.statusCode = 200;\n\nmsg.payload = {\n    status: \"success\",\n
shipment_id: shipmentId,\n    device_id: deviceId,\n    records_count:
records.length,\n    data: records\n};\n\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
```

```
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1230,
    "y": 1400,
    "wires": [
      [
        "f963244224e080a3"
      ]
    ]
  },
  {
    "id": "f963244224e080a3",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1430,
    "y": 1400,
    "wires": []
  },
  {
    "id": "b71e90d31be16a5d",
```

```
    "type": "http in",
    "z": "743c0a9528b8a2d7",
    "name": "GET Shipment Events",
    "url": "/api/v1/shipments/:id/events",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 600,
    "y": 1280,
    "wires": [
      [
        "e5f28953f1e20cd5"
      ]
    ]
  },
  {
    "id": "e5f28953f1e20cd5",
    "type": "file in",
    "z": "743c0a9528b8a2d7",
    "name": "Read Events CSV",
    "filename": "C:\\Users\\hp\\Desktop\\PROJECT IOT\\storage\\events.csv",
    "filenameType": "str",
    "format": "utf8",
    "chunk": false,
```

```
    "sendError": false,
    "encoding": "none",
    "x": 850,
    "y": 1280,
    "wires": [
      [
        "25a9e9c97e690d6f"
      ]
    ]
  },
  {
    "id": "25a9e9c97e690d6f",
    "type": "csv",
    "z": "743c0a9528b8a2d7",
    "name": "Parse Events CSV",
    "spec": "rfc",
    "sep": ",",
    "hdrin": "",
    "hdrout": "none",
    "multi": "mult",
    "ret": "\\r\\n",
    "temp": "timestamp,shipment_id,device_id,event_type,severity,description",
    "skip": "0",
    "strings": true,
    "include_empty_strings": "",
```

```
    "include_null_values": "",
    "x": 1110,
    "y": 1280,
    "wires": [
      [
        "8afdc382d6ddb1b2"
      ]
    ]
  },
  {
    "id": "8afdc382d6ddb1b2",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Filter Shipment Events",
    "func": "msg.headers = {\n    \"Content-Type\": \"application/json\"\n};\n\nlet shipmentId = msg.req.params.id;\nlet events = msg.payload;\n\nif (!Array.isArray(events)) {\n    msg.statusCode = 500;\n    msg.payload = {\n status: \"error\",\n        message: \"Invalid events data\"\n    };\n    return msg;\n}\n\nlet shipmentEvents = events.filter(row => {\n    return String(row.shipment_id).trim() === String(shipmentId).trim();\n});\n\nmsg.statusCode = 200;\n\nmsg.payload = {\n    status: \"success\",\n    shipment_id: shipmentId,\n events_count: shipmentEvents.length,\n    data: shipmentEvents\n};\n\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1320,
```

```
    "y": 1280,
    "wires": [
      [
        "f3db851e4fcf91fb"
      ]
    ]
  },
  {
    "id": "f3db851e4fcf91fb",
    "type": "http response",
    "z": "743c0a9528b8a2d7",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 1530,
    "y": 1280,
    "wires": []
  },
  {
    "id": "dfcf9efedf8f6967",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Auto Cooling Logic",
    "func": "let events = msg.events;\nlet data = msg.payload;\n\nif (!events ||
!events.includes(\"TEMPERATURE_CRITICAL\")) {\n    return null;\n}\n\nlet commandMsg
= {\n    topic: \"yemenfish/box/\" + data.device_id + \"/command\",\n    payload:
JSON.stringify({\n        command: \"COOLING_ON\",\n        source:
```

```
\"AUTOMATED_SAFETY_ENGINE\",\n        timestamp: new Date().toISOString()\n
})\n};\n\nreturn commandMsg;",

    "outputs": 1,

    "timeout": 0,

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 870,

    "y": 660,

    "wires": [

      [

        "b0494bdb336aafae",

        "f1d7e702c619e581"

      ]

    ]

  },

  {

    "id": "f1d7e702c619e581",

    "type": "debug",

    "z": "743c0a9528b8a2d7",

    "name": "debug 4",

    "active": true,

    "tosidebar": true,

    "console": false,

    "tostatus": false,
```

```
    "complete": "false",
    "statusVal": "",
    "statusType": "auto",
    "x": 1300,
    "y": 600,
    "wires": []
  },
  {
    "id": "a7755d475a63578d",
    "type": "function",
    "z": "743c0a9528b8a2d7",
    "name": "Prepare Email",
    "func": "let alertContent = msg.payload.content;\n\nlet newMsg = {};\nnewMsg.topic = \"🚨 إنذار طارئ: نظام YemenFish Guard\";\nnewMsg.payload = alertContent;\n\nreturn newMsg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 960,
    "y": 560,
    "wires": [
      [
        "091b27359849e2c8"
```

```
      ]
    ]
  },
  {
    "id": "091b27359849e2c8",
    "type": "e-mail",
    "z": "743c0a9528b8a2d7",
    "server": "smtp.gmail.com",
    "port": "465",
    "authtype": "BASIC",
    "saslformat": true,
    "token": "oauth2Response.access_token",
    "secure": true,
    "tls": true,
    "name": "hythmmran47@gmail.com",
    "dname": "",
    "x": 1180,
    "y": 700,
    "wires": []
  },
  {
    "id": "0b86c8486c5eb34c",
    "type": "mqtt-broker",
    "name": "",
    "broker": "127.0.0.1",
```

```
    "port": 1883,
    "clientid": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closeQos": "",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "",
    "willRetain": "",
    "willPayload": "",
    "userProps": "",
    "sessionExpiry": ""
  },
```

```
  {
    "id": "53959c28c2c680c3",
    "type": "ui-group",
    "name": "Current Box Status",
    "page": "2cd72be591b559d2",
    "width": 6,
    "height": 1,
    "order": 3,
    "showTitle": true,
    "className": "",
    "visible": "true",
    "disabled": "false",
    "groupType": "default"
  },
  {
    "id": "1b864c0addf77604",
    "type": "telegram bot",
    "botname": "YemenFish",
    "usernames": "",
    "chatids": "",
    "baseapiurl": "",
    "testenvironment": false,
    "updatemode": "polling",
    "pollinterval": 300,
    "usesocks": false,
```

```
    "sockshost": "",
    "socksprotocol": "socks5",
    "socksport": 6667,
    "socksusername": "anonymous",
    "sockspassword": "",
    "bothost": "",
    "botpath": "",
    "localbothost": "0.0.0.0",
    "localbotport": 8443,
    "publicbotport": 8443,
    "privatekey": "",
    "certificate": "",
    "useselfsignedcertificate": false,
    "sslterminated": false,
    "verboselogging": false
  },
  {
    "id": "45a0557bdb92c6ed",
    "type": "ui-group",
    "name": "Thermometer",
    "page": "2cd72be591b559d2",
    "width": 6,
    "height": 1,
    "order": 2,
    "showTitle": true,
```

```
    "className": "",
    "visible": "true",
    "disabled": "false",
    "groupType": "default"
  },
  {
    "id": "b3b683da1686f1cc",
    "type": "ui-group",
    "name": "Timeline",
    "page": "2cd72be591b559d2",
    "width": "12",
    "height": "1",
    "order": 1,
    "showTitle": true,
    "className": "",
    "visible": "true",
    "disabled": "false",
    "groupType": "default"
  },
  {
    "id": "2cd72be591b559d2",
    "type": "ui-page",
    "name": "Overview",
    "ui": "e45d95cf64f2dc9d",
    "path": "/page2",
```

```
    "icon": "home",
    "layout": "grid",
    "theme": "c76b1eb2de4fe9a6",
    "breakpoints": [
      {
        "name": "Default",
        "px": "0",
        "cols": "3"
      },
      {
        "name": "Tablet",
        "px": "576",
        "cols": "6"
      },
      {
        "name": "Small Desktop",
        "px": "768",
        "cols": "9"
      },
      {
        "name": "Desktop",
        "px": "1024",
        "cols": "12"
      }
    ],
```

```
    "order": 1,
    "className": "",
    "visible": "true",
    "disabled": "false"
  },
  {
    "id": "e45d95cf64f2dc9d",
    "type": "ui-base",
    "name": "YEMENFISH CONTROL CENTER",
    "path": "/dashboard",
    "appIcon": "",
    "includeClientData": true,
    "acceptsClientConfig": [
      "ui-notification",
      "ui-control"
    ],
    "showPathInSidebar": false,
    "headerContent": "page",
    "navigationStyle": "default",
    "titleBarStyle": "default",
    "showReconnectNotification": true,
    "notificationDisplayTime": 1,
    "showDisconnectNotification": true,
    "allowInstall": false
  },
```

```
  {
    "id": "c76b1eb2de4fe9a6",
    "type": "ui-theme",
    "name": "Default Theme",
    "colors": {
      "surface": "#fcfcfd",
      "primary": "#0d74ce",
      "bgPage": "#f9f9fb",
      "groupBg": "#ffffff",
      "groupOutline": "#d9d9e0"
    },
    "sizes": {
      "density": "default",
      "pagePadding": "12px",
      "groupGap": "12px",
      "groupBorderRadius": "4px",
      "widgetGap": "12px"
    }
  },
  {
    "id": "f5b369cb4a17558d",
    "type": "global-config",
    "env": [],
    "modules": {
      "@flowfuse/node-red-dashboard": "1.31.0",
```

```
      "node-red-contrib-telegrambot": "19.0.2",
      "node-red-node-email": "5.2.5"
    }
  }
```

Appendix C: Additional Material

Team Member	Contribution / Tasks Performed
Haitham Yahya Amran	Built MQTT integration and Core Engine logic.
Osama Mohammed Al-Shatbi	Designed HTTP APIs and Digital Passport algorithm.
Ahmed Abdo Al-Ghaili	Developed Dashboard UI, Alert Cooldowns, and Documentation.
Ahmed Mohammed Jahaf	Hardware simulation, testing, and deployment preparation.
Ibrahim Al-Khulaidi	Quality assurance, test cases validation, and report editing.